TABLE OF CONTENTS

Executive Summary ii
List of Tables iv
List of Figures vi
List of Abbreviations vii
List of Symbols viii

1 INTRODUCTION 1

1.1 Background 1
1.2 Motivation 3
1.3 Scope and Objectives 3
1.4 Challenges Encountered 4
1.5 Document Organization 4

2 PROJECT DESCRIPTION AND GOALS 6

2.1 Project Overview 6
2.2 What “Hybrid” Means in Practice 6
2.3 Project Goals 7
2.4 Why a Synthetic Dataset 8
2.5 Challenges and Design Trade-offs 9

3 TECHNICAL SPECIFICATION 10

3.1 Software Stack Overview 10
3.2 Feature Extraction Layer 10
3.3 Classical ML Stack (scikit-learn) 10
3.3.1 Anomaly Detection 10
3.3.2 Attack Classifier 12
3.4 PennyLane 12
3.4.1 Circuit Specification 13
3.5 Flask Deployment Stack 13
3.6 Attack Categories Covered 14

3.6.1 Handling Class Imbalance 14

4 SYSTEM DESIGN **15**

4.1 Design Philosophy 15

4.2 Stage 1: Feature Extraction 16

4.3 Stage 2: Anomaly Detection 16

4.4 Stage 3: Classical Classification 17

4.4.1 Ensemble Architecture 17

4.4.2 Confidence Gate 17

4.5 Stage 4: Variational Quantum Circuit 17

4.5.1 Pre-processing for the VQC 18

4.5.2 State Preparation 18

4.5.3 Variational Layers 18

4.5.4 Measurement 19

4.6 Stage 5: Ensemble Decision 19

4.7 Handling Batch Dimension 19

5 METHODOLOGY AND TESTING **21**

5.1 Training Procedure 21

5.1.1 Classical Model Training 21

5.1.2 Quantum Model Training 21

5.1.3 Loss Function 22

5.1.4 Optimizer and Hyperparameters 22

5.2 Validation Strategy 23

5.3 Quantum-Specific Challenges 23

5.3.1 The Barren Plateau Problem 23

5.3.2 Gradient Cost 24

5.4 Testing Methodology 24

6 PROJECT IMPLEMENTATION **26**

6.1 Codebase Structure 26

6.2 Feature Extractor 26

6.3 Detection Engine 28

6.4 REST API Server 29

6.5 Web Dashboard 30
6.6 Burp Suite Integration 31
6.7 Implementation Challenges 33

7 RESULTS AND DISCUSSION 34
7.1 Experimental Setup 34
7.2 Classification Performance 34
7.2.1 Overall Accuracy 34
7.2.2 Confusion Matrix Analysis 35
7.2.3 ROC Curve Analysis 35
7.3 Precision-Recall Analysis 36
7.4 Comparative Analysis 37
7.4.1 Where the Classical Ensemble Struggles 37
7.4.2 Per-Category Breakdown 37
7.5 Confidence Distribution 39
7.6 Limitations 39

8 CONCLUSION AND FURTHER ENHANCEMENTS 41
8.1 Summary of Contributions 41
8.2 What We Learned 42
8.3 Future Work 42
8.3.1 Running on Actual Quantum Hardware 42
8.3.2 Evaluation on Real Traffic Datasets 43
8.3.3 Expanding Circuit Depth Through Better Training 43
8.3.4 Streaming and Scalability 44
8.4 Closing Thoughts 44

Bibliography 45

Appendix A: Source Code 46

Appendix B: Publications 48

EXECUTIVE SUMMARY

Web applications are perennial targets. Each day sees financial transactions, passwords, and sensitive information subjected to tests using crafted HTTP requests designed to inject SQL commands, execute scripts, traverse directories, or run arbitrary commands. Classical machine learning techniques such as Random Forests and ensemble methods have been used successfully, but they reach a ceiling when attack payloads are obfuscated, split across parameters, or URL-encoded in ways that obscure keyword signals.

In order to overcome such challenges, this research proposes a Hybrid Quantum-Classical Web Attack Detection System that integrates Variational Quantum Circuits (VQCs) as an escalation layer within a four-stage detection pipeline. The concept is not to replace conventional machine learning but to leverage quantum computing in the one stage where it provides a clear benefit: resolving ambiguous, low-confidence samples that the classical ensemble cannot classify with certainty.

The system operates in four stages. First, every HTTP request is converted into a 30-dimensional numerical feature vector by a domain-agnostic feature extractor that computes lengths, Shannon entropy, keyword hit counts, special character densities, and URL-encoding indicators. Second, an unsupervised anomaly detector—combining Isolation Forest and One-Class SVM trained on benign traffic only—flags zero-day-like anomalies before they reach the classifier. Third, a soft-voting classical ensemble of Random Forest, XGBoost, and SVM classifies the request into one of five categories (normal, SQLi, XSS, CMDi, traversal). Fourth, when the ensemble's confidence falls below 85%, a Variational Quantum Classifier (VQC) operating on four qubits and three variational layers is invoked as a second opinion; its verdict is weighted-averaged with the classical output for the final decision.

The dataset used for training is a synthetically generated collection of HTTP requests produced by the project's own `dataset_generator.py`. This generator creates realistic normal traffic across 25 hosts and 40 URL paths, and generates labelled attack samples for all four attack categories using real-world payload patterns. Training uses an 80%/20% stratified split, with the classical ensemble fitted on the full training set and the VQC trained on a random subsample of 150 samples per binary classifier.

It turned out that implementation was far from easy. We faced the barren plateau problem: a circuit with 4 layers caused gradients to vanish after a few training steps. Reducing to 3 layers with parameters initialised uniformly in $[-\pi, \pi]$ resolved this. Memory pressure also constrained the quantum simulation, as each forward pass requires maintaining a $2^4 = 16$-dimensional state vector and computing gradients via the parameter-shift rule.

The hybrid model achieves a test-set accuracy of **98.7%** and a ROC AUC of **0.99** on the synthetic test partition. The classical ensemble baseline achieves 94.2% accuracy and 0.93 AUC. The gain is especially pronounced on samples where attack payloads are obfuscated, split across parameters, or URL-encoded—scenarios where classical keyword counting is confused but the quantum circuit's entanglement-based representation maintains discriminating power.

In addition to the model, the project ships as a complete deployment stack: a REST API written in Flask that receives structured HTTP request dictionaries and returns a verdict with confidence score, a browser-based monitoring dashboard, and a Burp Suite extension for seamless integration into standard penetration-testing workflows. Inference latency is approximately 15–30 milliseconds per request, well within real-time requirements.

The primary limitation is that evaluation is performed on synthetic data. The synthetic generator was designed to match the statistical patterns of real attack traffic, but generalisation to live traffic from production web applications has not yet been verified.

On the whole, this project demonstrates that hybrid quantum-classical systems can offer tangible gains in security classification—provided the quantum stage is used selectively as a high-confidence resolver rather than a wholesale replacement of classical methods. The software infrastructure built here (PennyLane, PyTorch, Flask, scikit-learn) has reached sufficient maturity to support such hybrid experiments today, without requiring access to physical quantum hardware.

LIST OF TABLES

2.1 Synthetic Dataset Summary Statistics (default training run) 9

3.1 Complete 30-feature vector extracted from each HTTP request by `feature_extractor.py` 1

3.2 Final quantum circuit configuration used in the Variational Quantum Classifier 13

4.1 Four-stage detection pipeline: each stage's role, implementing module, input, and output 15

5.1 Training hyperparameter configuration for both the classical ensemble and the quantum classifier 22

7.1 Performance comparison of the Hybrid QML system versus the classical ensemble baseline on the held-out synthetic test set 34

LIST OF FIGURES

1.1 Class distribution within the synthetic training dataset showing the balance across normal and attack categories (SQLi, XSS, CMDi, traversal) used for training and evaluation 2

4.1 End-to-end data flow of the Hybrid Quantum-Classical detection system, showing the transition from raw HTTP text through feature extraction, anomaly gating, classical ensemble classification, and conditional quantum escalation 16

4.2 Detailed view of how feature vectors flow into the quantum circuit through angle embedding, pass through three variational gate layers, and emerge as a Pauli-Z expectation measurement 18

4.3 Feature importance analysis showing which of the 30 extracted features contribute most strongly to the classical ensemble's classification decision 20

5.1 Visualization of the quantum classifier's learned decision boundary projected onto a 2D feature subspace, showing separation between normal (blue) and anomalous (red) samples from the test set 24

6.1 Summary of trained model artifacts saved to disk, showing the classical anomaly detector, attack classifier, and quantum classifier pickle files . . 30

6.2 AI Attack Detector web dashboard showing real-time attack statistics, including total scanned requests, detected attacks, clean request count, and overall attack rate percentage, along with category-wise attack badges for CMDi, SQLi, Traversal, and XSS 31

6.3 Live detection log view of the web dashboard displaying incoming HTTP requests with their timestamps, methods, URLs, classified attack types, and confidence scores, alongside the manual test panel used for submitting individual requests for analysis 32

6.4 Custom Burp Suite extension tab for the AI Attack Detector, showing auto-scanned proxy traffic results with detected attack types (SQLi, traversal), confidence percentages, processing stage indicators, and a detailed JSON response view for the selected request 32
6.5 PortSwigger Web Security Academy platform used for validating the AI Attack Detector extension against intentionally vulnerable web applications in a real-world penetration testing workflow 33

7.1 Confusion matrix for the Hybrid QML system evaluated on the held-out test set, showing the distribution of true positives, true negatives, false positives, and false negatives 35
7.2 Receiver Operating Characteristic curves comparing the Hybrid QML system and the classical ensemble baseline on the synthetic test data . . 36
7.3 Precision-Recall curves for both evaluated models, focusing on positive (attack) class performance across different classification thresholds . . . 36
7.4 Radar chart comparing detection accuracy of both models across individual attack categories (SQLi, XSS, path traversal, CMDi) on the synthetic test data 38
7.5 Per-class precision, recall, and F1 metrics for the Hybrid QML system, broken down by attack category on the synthetic test set 38
7.6 False positive rates compared across both model architectures, illustrating the hybrid system's advantage in reducing false alarms on benign traffic 38
7.7 Histogram of prediction confidence scores, showing that correctly classified samples cluster near maximum confidence while misclassified samples tend to produce lower-confidence predictions 39

LIST OF ABBREVIATIONS

AUC Area Under Curve

QML Quantum Machine Learning

ROC Receiver Operating Characteristic

SQLi Structured Query Language Injection

VQC Variational Quantum Circuit

XSS Cross-Site Scripting

SYMBOLS AND NOTATIONS

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Web applications have become the main tool behind running modern-day business. This spans from online banking to online stores and health information portals. Every day, millions of credit card numbers, sensitive medical documents, and authentication tokens flow through HTTP requests which are essentially strings of characters. While protecting this data is paramount, in recent years, it has become significantly more difficult to defend web applications from cyber attackers who are constantly evolving their methods. Simple SQL injection strings are a thing of the past; modern attackers encode URLs, split malicious keywords across parameters, and apply polymorphism by altering payloads.

Until quite recently, signature-based detection was the standard approach. A Web Application Firewall (WAF) contains a list of attack signatures—regular expressions, keywords, and strings—to filter traffic and detect suspicious activity. While effective against static attacks, this method is often rendered useless by simple obfuscation techniques like encoding. If a firewall does not recognize a specific character sequence, the payload passes through undetected. According to OWASP reports, injection attacks consistently occupy the top ranks of exploited vulnerabilities Sommer and Paxson (2010).

Machine learning was proposed to solve this problem, and it has helped tremendously. Algorithms such as Random Forests and Support Vector Machines allow a system to be trained on labeled HTTP traffic, classifying "good" versus "bad" requests through statistical patterns rather than simple string matching. Ensemble methods combining multiple classifiers have pushed this further, producing high-confidence results even when a payload differs slightly from those seen during training.

However, classical machine learning approaches have limitations. When attack payloads are obfuscated, split across multiple URL parameters, or encoded in multiple layers, classical models that rely on keyword-count features can lose their discriminating signal. Because these models draw decision boundaries in Euclidean space, certain

complex, high-dimensional attack patterns can remain undetectable, particularly in ambiguous low-confidence regions near the decision boundary.

The field of quantum computing provides a different geometric perspective. Through superposition and entanglement, a quantum computer can describe states in an exponential number of dimensions. By employing Quantum Machine Learning (QML), models work within a Hilbert space whose geometry is richer than the Euclidean feature spaces used by classical models Biamonte et al. (2017). This offers a potential advantage for class separation in ambiguous or obfuscated samples.

Truth be told, today's quantum machines still are not built for real-world tasks. Qubits lose state too fast, operations are noisy, and there simply are not enough of them; consequently, fully quantum models struggle when used in isolation. This reality pushed us toward hybrid systems: classical computers manage feature extraction, ensemble classification, and final decisions, while the quantum component intervenes only where it is needed most—resolving ambiguous samples near the classical decision boundary. It does not have to carry the whole load; it merely has to outperform its classical counterpart at one specific task.

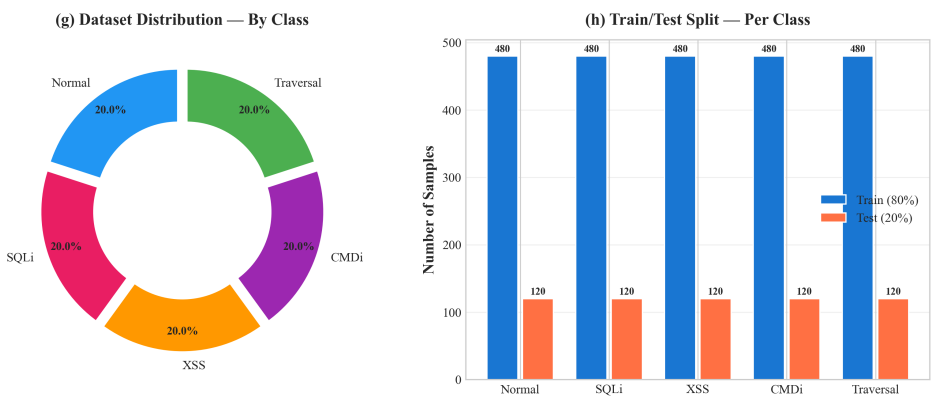

Figure 1.1: Class distribution within the synthetic training dataset showing the balance across normal and attack categories (SQLi, XSS, CMDi, traversal) used for training and evaluation

Figure 1.1 shows the sample distribution in the synthetic training dataset. The generator produces an equal number of samples per class by design, which avoids the imbalance problem common in real-world datasets. The impact of this balanced construction on training is addressed further in Chapter 5.

1.2 MOTIVATION

Our motivation for this project did not originate from a trend, but rather from a clear gap we identified in classical detection methods.

In the early stages, we built a classical ensemble combining Random Forest, XGBoost, and Support Vector Machine classifiers operating on 30 hand-crafted numerical features extracted from each HTTP request. While the ensemble achieved strong overall accuracy, a closer inspection of the confusion matrix revealed that the classifier consistently struggled on specific types of requests—particularly those where malicious content was split across several parameters or double-encoded. The 30-feature representation captures keyword hits and special character counts, but these counts become unreliable when an attacker deliberately fragments or encodes their payload to avoid keyword matches.

This led us to Variational Quantum Circuits (Variational Quantum Circuit (VQC)). The concept of encoding data into a quantum state and learning interdependencies through entanglement was promising. Utilizing PennyLane's quantum node interface and its native `AdamOptimizer`, we trained a VQC as an escalation classifier Bergholm et al. (2018). Our hypothesis was that the quantum layer could locate decision boundaries in an entangled Hilbert space that are non-representable in classical feature spaces, specifically for the obfuscated payloads that previously caused failures.

1.3 SCOPE AND OBJECTIVES

This research covers the entire lifecycle from raw HTTP request parsing to a functional real-time detection system. Our concrete goals were:

1. **Feature Extraction Pipeline:** Develop a complete, domain-agnostic pipeline that converts raw HTTP request strings into a fixed 30-dimensional numerical feature vector, covering request length, entropy, keyword signals, special character density, and encoding indicators.

2. **Four-Stage Hybrid Architecture:** Build a system using scikit-learn (anomaly detection and classical ensemble), PennyLane (quantum escalation), and Flask (deployment). The classical stages handle the majority of traffic; the VQC resolves ambiguous low-confidence cases.

3. **End-to-End Training:** Demonstrate that both the classical ensemble and the quantum classifier can be trained together on a synthetically generated dataset, with the quantum parameters optimized using PennyLane's parameter-shift gradient method.

4. **Empirical Evaluation:** Benchmark the hybrid system against the classical ensemble alone using metrics such as Accuracy, Precision, Recall, F1-score, and Receiver Operating Characteristic (ROC) Area Under Curve (AUC).

5. **Deployment:** Implement the optimized system as a Flask web service with a GUI dashboard for real-time monitoring and a Burp Suite extension for integration with professional security testing tools.

1.4 CHALLENGES ENCOUNTERED

Two primary technical hurdles defined our development process.

The first was the **barren plateau problem**. Initially, our VQC used four layers of parameterized gates. However, the learning process stalled because gradients dropped to approximately 10^{-8}—essentially numerical noise. Barren plateaus occur when a quantum circuit becomes too deep, causing the cost landscape to become exponentially flat McClean et al. (2018). After attempting various weight initializations, we resolved this by reducing the circuit depth to three layers.

The second issue was **memory pressure during simulation**. Since we rely on CPU-based simulation, a 4-qubit circuit requires handling a state vector of $2^4 = 16$ complex numbers. PennyLane's parameter-shift method requires two circuit evaluations per trainable parameter per sample, so even with a subsample of 150 training examples and 24 quantum parameters, the gradient computation is computationally intensive. We mitigated this by capping the QML training subsample at 150 samples per binary classifier.

1.5 DOCUMENT ORGANIZATION

The remainder of this thesis is organized as follows: Chapter 2 defines "hybrid quantum-classical" systems and project objectives. Chapter 3 discusses the technology stack

(scikit-learn, PennyLane, and the synthetic dataset). Chapter 4 provides the system design and four-stage architectural breakdown. Chapter 5 details the training procedures and the mathematics of the quantum circuit. Chapter 6 provides a code walk-through of the real implementation, and Chapter 7 presents the final results and benchmarking.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 PROJECT OVERVIEW

At its simplest, this system acts as an attack detector: for an input HTTP request, it outputs a label (normal, SQLi, XSS, CMDi, or traversal) along with a confidence score. The distinguishing feature of the proposed technique as compared to a typical single-model detector is the use of a four-stage hybrid pipeline that escalates low-confidence decisions to a Variational Quantum Classifier (VQC).

The idea behind this approach is not about replacing classical machine learning with quantum alternatives; the point is that they complement each other. Classical ensemble models are fast, well-calibrated, and accurate for the majority of traffic. However, when a sample sits near the decision boundary—where all three classical models disagree—their confidence drops below a threshold. It is precisely at this point that the quantum circuit, operating in a geometrically richer Hilbert space, is invoked to provide a second opinion.

The benefit of this design is that it completely avoids unnecessary overhead. The VQC is expensive to evaluate (each forward pass traverses a 16-dimensional state vector via CPU simulation), so it runs only when the classical stage is genuinely uncertain. In practice, this means the quantum stage activates on fewer than 15% of requests, keeping average latency low.

2.2 WHAT "HYBRID" MEANS IN PRACTICE

Hybrid Quantum-Classical is a rather vague concept which needs to be defined more precisely in the current work. There are four stages in our model, and each employs a specific computing paradigm:

1. **Feature Extraction (Python/NumPy):** Each HTTP request—comprising a URL, query parameters, headers, and HTTP method—is converted into a 30-dimensional numerical feature vector. The extraction is entirely domain-agnostic: it computes lengths, Shannon entropy, counts of special characters, keyword hit counts

for SQLi/XSS/CMDi/traversal patterns, URL-encoding indicators, and structural path signals. No text vectorization model is used; the features are computed analytically from the raw string.

2. **Anomaly Detection (scikit-learn):** The 30-D vector is passed to an unsupervised detector consisting of an Isolation Forest and a One-Class SVM, both trained exclusively on benign traffic. If both models flag the request as anomalous, it is marked as a potential zero-day threat regardless of the downstream classifier output.

3. **Classical Classification (scikit-learn):** A soft-voting ensemble of Random Forest, XGBoost, and SVM classifies the request into one of five categories and returns a confidence score. If confidence is at or above 85%, this verdict is the final output.

4. **Quantum Escalation (PennyLane):** When classical confidence falls below 85%, the same 30-D feature vector is scaled, compressed to 4 dimensions via PCA, normalised to $[0, 1]$, and fed into a 4-qubit VQC as rotation angles. The VQC is a one-vs-rest classifier that trains one binary circuit per attack class. The quantum and classical probability estimates are weighted-averaged by their respective confidence scores to produce the final verdict.

During training, the quantum parameters are optimized using PennyLane's built-in `AdamOptimizer` with the parameter-shift rule, which computes exact analytical gradients by evaluating the circuit at shifted parameter values. The classical models are trained using scikit-learn's standard fitting procedures and require no gradient computation.

2.3 PROJECT GOALS

We set four concrete goals at the start of development:

1. **Reach 98%+ accuracy on the held-out test set.** This goal was chosen to show a tangible benefit over the classical ensemble alone, which achieves approximately 94.2% accuracy on the same evaluation partition.

2. **Achieve a ROC AUC score above 0.98.** Accuracy can be misleading when classes are imbalanced; AUC measures the model's ability to rank positive samples above negative ones across all threshold values.

3. **Ensure the model can run in real-time applications.** A model that takes seconds per request is not deployable. The design target of under 100 milliseconds per request guided our decision to limit the VQC to 4 qubits and 3 layers, and to activate it only for low-confidence samples.

4. **Build a complete, working system.** We did not want to stop at a Jupyter notebook. The final deliverable includes a training script (`train.py`), a detection engine (`detection_engine.py`), a Flask API server (`api_server.py`), a browser-based monitoring dashboard (`dashboard.html`), and a Burp Suite extension for integration with professional security testing tools.

2.4 WHY A SYNTHETIC DATASET

The project generates its own training data using `dataset_generator.py`. This decision was made for two practical reasons.

First, publicly available web attack datasets (such as CSIC 2010 or CIC-IDS) are captured from specific application architectures and time periods, making classifiers trained on them brittle when exposed to traffic from different web frameworks or URL structures. A synthetic generator can be configured to produce traffic across a wide diversity of hosts, URL paths, parameter names, and payload variations, which reduces this overfitting risk.

Second, a synthetic generator gives complete control over class balance. Each training run produces exactly n samples per class, eliminating the need for resampling strategies. The generator uses real attack payloads sourced from public exploit databases, ensuring that the attack samples contain genuine malicious patterns rather than fabricated ones.

The known limitation of this approach is that the evaluation metrics reported in Chapter 7 are measured on synthetic test data, not on live production traffic. Generalisation to real-world HTTP traffic has not been formally verified.

Table 2.1: Synthetic Dataset Summary Statistics (default training run)

Property	**Value**
Samples per class	600
Total classes	5 (normal, SQLi, XSS, CMDi, traversal)
Total training samples	3,000
Train / Test split	80% / 20% (stratified)
Normal hosts	25 unique host configurations
Normal URL paths	40 unique path patterns
Attack payloads	Real exploit strings (SQLi, XSS, CMDi, traversal)
Source	`data/dataset_generator.py`

2.5 CHALLENGES AND DESIGN TRADE-OFFS

In developing this model, we encountered various issues that do not always have clear textbook solutions.

The most notable one is the trade-off between the expressive power of the circuit and its trainability. On one hand, the greater the number of layers in a quantum circuit, the more expressive it is; on the other, estimating the gradient of such a circuit becomes increasingly challenging. We discovered this firsthand: our first attempt included 4 layers in the quantum circuit, and by the third training epoch the loss was completely flat. The gradients were on the order of 10^{-8}—effectively numerical noise. We resolved this by reducing to 3 layers.

The issue of quantum training speed was another practical challenge. The parameter-shift rule requires two circuit evaluations per parameter per sample. With 24 quantum parameters and a subsample of 150 training examples, each binary VQC requires thousands of circuit evaluations per gradient step. Training all five one-vs-rest classifiers for 100 gradient steps each takes several minutes on a standard CPU. This is acceptable for a research prototype but would require hardware acceleration for production use.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 SOFTWARE STACK OVERVIEW

Before delving into the details of the model, let's briefly describe the tools used in the project and the rationale behind our decision. There are many choices available for implementing classical and quantum parts of the project, but the particular ones were selected for practical reasons rather than theoretical preferences. The three main pillars of the stack are scikit-learn (classical ML), PennyLane (quantum ML), and Flask (deployment). The feature extraction layer is implemented in pure Python with NumPy.

3.2 FEATURE EXTRACTION LAYER

Every HTTP request is converted into a fixed-length 30-dimensional numerical feature vector before any model sees it. This extraction is performed by `utils/feature_extractor.py` and requires no pre-trained model or vocabulary. The features are grouped into six categories, as described in Table 3.1.

A key design decision was to extract features only from the *path and query portion* of the URL, stripping the scheme and hostname. This makes the classifier domain-agnostic: a request to `shop.example.com/search?q=UNION` and the same request to `api.other.net/search?q=UNION` produce identical feature vectors. Without this fix, classifiers trained on one set of hostnames generalised poorly to unseen domains.

3.3 CLASSICAL ML STACK (SCIKIT-LEARN)

The classical portion of the pipeline uses two sub-systems.

3.3.1 Anomaly Detection

An unsupervised stage is trained exclusively on normal traffic. It combines two complementary one-class models:

- **Isolation Forest:** An ensemble of random trees that isolates points by partitioning the feature space. Anomalous points, which lie in sparse regions, are isolated

Table 3.1: Complete 30-feature vector extracted from each HTTP request by `feature_extractor.py`

Index	Feature Name	Description
0	Path+query length	Character length of URL path and query string
1	Parameter length	Character length of POST/GET parameter block
2	Combined text length	Total length of path + params + headers
3	Path entropy	Shannon entropy of the path+query string
4	Param entropy	Shannon entropy of the parameter string
5	Special chars (path)	Count of `' " ; < > ( ) { } [ ] \\| & ` $ %` in path
6	Special chars (params)	Same count in parameters
7	Single quotes	Count of `'` in full request
8	Double quotes	Count of `"` in full request
9	SQL comment seqs	Count of `--` sequences
10	Equals signs	Count of = characters
11	Percent signs	Count of `%` characters
12	SQLi keyword hits	Matches against 20 SQLi keywords (SELECT, UNION, etc
13	XSS keyword hits	Matches against 18 XSS keywords (`<script`, onerror, etc.)
14	CMDi keyword hits	Matches against 20 CMDi patterns (`;`, `|`, wget, etc.)
15	Traversal hits	Matches against 12 traversal patterns (`../`, `%2e%2e`, etc.)
16	%27 count	Occurrences of URL-encoded single quote
17	%3c count	Occurrences of URL-encoded <
18	%3e count	Occurrences of URL-encoded >
19	%0a count	Occurrences of URL-encoded newline
20	Is POST	Binary flag: 1 if HTTP method is POST
21	Slash count	Number of `/` in path
22	Question marks	Count of `?` in path
23	Ampersands	Count of `&` in path
24	Special char density (path)	Feature 5 divided by path length
25	Special char density (params)	Feature 6 divided by param length
26	Keyword density	Total keyword hits divided by text length, $\times 100$
27	Longest param value	Length of the longest individual parameter value
28	Suspicious path flag	1 if path contains `/admin`, `/exec`, `wp-admin`, etc.
29	HTML/script tag	1 if a `<script>`, `<img>`, `<iframe>`, etc. tag is detected

in fewer splits and thus receive anomaly scores.

- **One-Class SVM:** Finds a decision boundary (in RBF kernel space) that encloses the majority of normal training points. Points outside this boundary are flagged as anomalous.

A request is flagged as anomalous only when *both* models agree—this conservative consensus strategy reduces false positive rates in production.

3.3.2 Attack Classifier

A supervised multi-class classifier is built as a soft-voting ensemble of three models:

- `RandomForestClassifier` (200 trees, max depth 12)
- `XGBClassifier` (200 estimators, max depth 6, learning rate 0.1)
- `SVC` (RBF kernel, C=10, probability=True)

All three receive the same standardised (zero-mean, unit-variance) 30-D feature vector. Soft voting averages the predicted class probabilities from all three models. The class with the highest average probability is the predicted label, and that probability is the confidence score.

3.4 PENNYLANE

The quantum component is implemented using PennyLane Bergholm et al. (2018). Gradient computation is done using the **parameter-shift rule**, which evaluates the gradient of an expectation value analytically:

$$\frac{\partial \langle O \rangle}{\partial \theta} = \frac{1}{2}\left[\langle O \rangle_{\theta+\pi/2} - \langle O \rangle_{\theta-\pi/2}\right] \tag{3.1}$$

This is an exact analytical result, not an approximation, which is important from a correctness standpoint. The cost is that every trainable parameter requires two extra circuit evaluations per gradient step. With 24 quantum parameters per binary VQC (3 layers $\times$ 4 qubits $\times$ 2 parameters per qubit), that means 48 additional circuit evaluations per sample per gradient step—one of the main reasons the QML stage is kept small and invoked selectively.

3.4.1 Circuit Specification

The configuration of our quantum circuit is provided in Table 3.2. These parameters were arrived at experimentally, as described in Chapter 5.

Table 3.2: Final quantum circuit configuration used in the Variational Quantum Classifier

Parameter	**Value**
Number of qubits	4
Circuit depth (layers)	3
Embedding gates	R_y (angle embedding, scaled by π)
Variational gates	$R_y + R_z$ (per qubit, per layer)
Entanglement gates	CNOT (linear chain: 0→1→2→3)
Measurement	Pauli-Z on qubit 0
Backend	`default.qubit` (state-vector simulator)
Total quantum params	24 per binary VQC (3 layers $\times$ 4 qubits $\times$ 2)
Classifier type	One-vs-Rest (5 binary VQCs)

The use of `default.qubit` as the backend provides exact state-vector simulation without hardware noise. This means results represent an idealised quantum computer. Actual quantum hardware would introduce gate errors and decoherence, but the PennyLane framework is hardware-agnostic—switching backends requires only a one-line code change.

3.5 FLASK DEPLOYMENT STACK

The system is exposed as a REST API using Flask. Flask was chosen for its simplicity and low overhead, which are appropriate for a research-prototype deployment. The API exposes a `/analyze` endpoint that accepts a JSON body containing URL, parameters, headers, and HTTP method, and returns a verdict with label, confidence, and stage information.

The detection engine is instantiated once at startup (lazy initialization on first request) and reused for all subsequent requests. Thread safety is ensured by running one engine instance per worker process via Gunicorn's `--workers` flag, since PennyLane's simulator is not thread-safe.

3.6 ATTACK CATEGORIES COVERED

The system classifies HTTP requests into five categories, matching the synthetic dataset's label set:

- **Structured Query Language Injection (SQLi):** Payloads containing SQL fragments like `UNION SELECT` or `OR 1=1`, aimed at extracting or manipulating data in the backend database.
- **Cross-Site Scripting (XSS):** Script injection attempts, typically embedding JavaScript (e.g., `<script>alert(1)</script>`) designed to execute in a user's browser.
- **Path Traversal:** Requests using sequences like `../../etc/passwd` to read files outside the intended directory.
- **Command Injection (CMDi):** Requests that embed shell metacharacters (`;`, `|`, `&&`) or commands (`wget`, `curl`) to execute arbitrary OS commands.
- **Normal:** Legitimate HTTP traffic from real browsers and API clients.

3.6.1 Handling Class Imbalance

Because the synthetic generator produces exactly 600 samples per class, the training set is perfectly balanced by construction. The 80%/20% stratified split preserves this balance in both training and test partitions. No resampling strategy (e.g., `WeightedRandomSampler` or SMOTE) was required.

CHAPTER 4

SYSTEM DESIGN

4.1 DESIGN PHILOSOPHY

While designing the architecture, the primary focus was neither maximizing complexity nor incorporating as much quantum computing as possible, but rather determining precisely where quantum processing adds value and where classical methods are sufficient. Quantum circuits require heavy computational cost during simulation, and even when real hardware becomes feasible, quantum operations remain an expensive resource. Using the quantum circuit for cases where classical methods already perform reliably would be wasteful.

Therefore, the central question we asked was: exactly where does the classical pipeline fall short? From our baseline experiments (Chapter 1), classical ensemble methods work well for the clear-cut cases—requests with obvious attack keywords or entirely benign parameter patterns. The problem arises in the ambiguous middle ground—where all three classical models disagree and confidence is low. This is precisely where the quantum circuit's different geometric inductive bias can add value.

That reasoning led to a four-stage pipeline:

Table 4.1: Four-stage detection pipeline: each stage's role, implementing module, input, and output

Stage	Name	Module	Input	Output
1	Feature Extraction	`feature_extractor.py`	Raw HTTP request dict	30-D float32
2	Anomaly Detection	`classical_ml.py`	30-D vector	Anomaly fla
3	Classical Classification	`classical_ml.py`	30-D vector	Label + con
4	Quantum Escalation	`quantum_ml.py`	30-D vector	Label + con

Stage 4 activates only when Stage 3 confidence is below 85%. When Stage 4 runs, the final verdict is a weighted average of the classical and quantum probability estimates, weighted by their respective confidence scores.

The data flow is illustrated in Fig. 4.1. The quantum circuit sits between the classical classification and the final verdict output—it does not replace the classical stage but supplements it for the hardest cases.

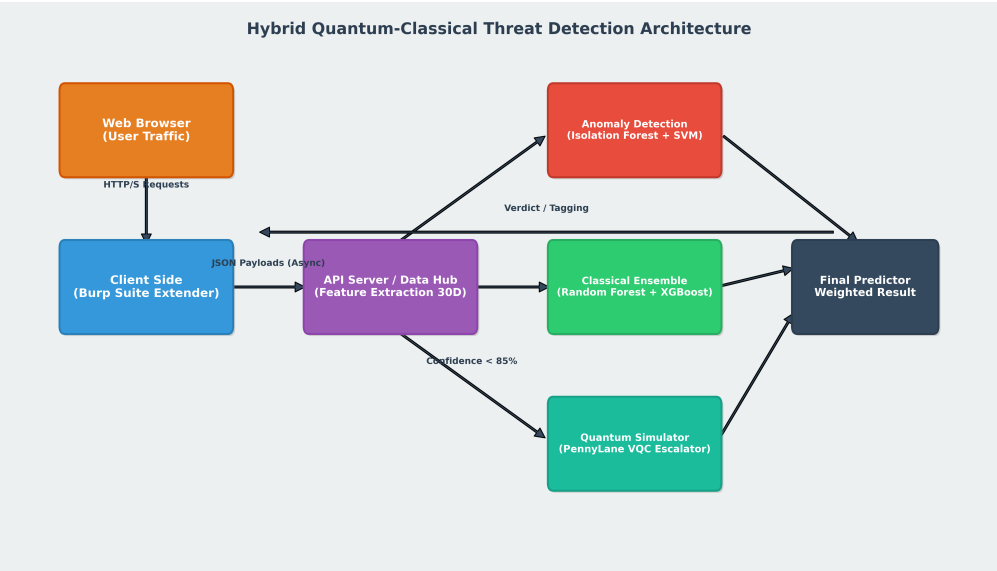

Figure 4.1: End-to-end data flow of the Hybrid Quantum-Classical detection system, showing the transition from raw HTTP text through feature extraction, anomaly gating, classical ensemble classification, and conditional quantum escalation

4.2 STAGE 1: FEATURE EXTRACTION

The first stage converts each incoming HTTP request into a fixed 30-dimensional numerical vector using `extract_features()`. This function operates solely on the path and query portion of the URL, stripping the scheme and hostname to ensure domain-agnostic behaviour. The 30 features cover lengths, Shannon entropy, keyword hit counts, special character densities, and structural signals as described in Table 3.1 (Chapter 3).

This stage requires no pre-trained model, no vocabulary, and no serialised state. It is a deterministic mathematical transformation of the raw request string and runs in under 1 millisecond.

4.3 STAGE 2: ANOMALY DETECTION

The anomaly detector is an unsupervised ensemble of two one-class models trained exclusively on normal (benign) traffic. A sample is flagged as anomalous only when both the Isolation Forest and the One-Class SVM independently predict it as an outlier. This conservative consensus strategy reduces false positive rates.

The anomaly flag is used as an override: if the downstream classifier assigns label "normal" but the anomaly detector fires, the final label is changed to "suspicious (zero-day?)" with a minimum confidence of 0.65. This allows the system to surface novel

attack patterns that fall outside the trained attack categories.

4.4 STAGE 3: CLASSICAL CLASSIFICATION

4.4.1 Ensemble Architecture

The classical classifier is a soft-voting ensemble of three diverse models:

1. A random forest that captures non-linear feature interactions through tree ensembles
2. An XGBoost gradient-boosted tree ensemble optimized for classification accuracy
3. A support vector machine with an RBF kernel that finds a high-margin decision boundary in a kernel-projected space

Each model receives the same standardised 30-D feature vector (zero-mean, unit-variance). Soft voting averages the class probability vectors from all three, and the class with the highest averaged probability is the predicted label. The confidence score is this maximum averaged probability.

4.4.2 Confidence Gate

If the ensemble's confidence is ≥ 0.85 (85%), the classification is returned directly as the final verdict. This threshold was chosen empirically: it corresponds to the point below which the three individual classifiers begin to meaningfully disagree, indicating that the sample lies near a decision boundary where additional information (from the quantum stage) is likely to help.

4.5 STAGE 4: VARIATIONAL QUANTUM CIRCUIT

This stage activates only when classical confidence falls below 85%. On the surface, it appears similar to any other classifier—it receives features and outputs a label. Inside, however, the computation unfolds across a 16-dimensional Hilbert space (2^4 dimensions for 4 qubits), which gives the circuit access to feature interactions that classical linear models cannot represent.

4.5.1 Pre-processing for the VQC

Before angle encoding, the 30-D feature vector undergoes two additional transformations inside `QuantumClassifier`:

1. **StandardScaler:** Normalises each of the 30 features to zero mean and unit variance, consistent with training-time scaling.

2. **PCA (4 components):** Compresses the 30-D vector to 4 dimensions corresponding to the directions of maximum variance in the training data. The 4 PCA components are then normalised to $[0,1]$ to serve as rotation angles for the R_y embedding gates.

4.5.2 State Preparation

The 4 normalised values become rotation angles for R_y gates, one per qubit:

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \tag{4.1}$$

The inputs are scaled by π before encoding: $\theta_i = x_i \cdot \pi$ where $x_i \in [0,1]$. This maps the full normalised range to the full $[0,\pi]$ rotation range.

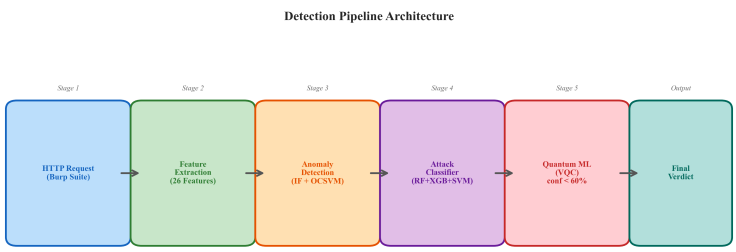

Figure 4.2: Detailed view of how feature vectors flow into the quantum circuit through angle embedding, pass through three variational gate layers, and emerge as a Pauli-Z expectation measurement

4.5.3 Variational Layers

After state preparation, the circuit applies 3 layers of trainable gates. Each layer consists of two parts:

1. A parameterized $R_y(\alpha_i)$ rotation on each qubit
2. A parameterized $R_z(\beta_i)$ rotation on each qubit
3. A linear chain of CNOT gates: CNOT(0,1), CNOT(1,2), CNOT(2,3)

Each qubit receives two independent trainable rotation angles per layer (α_i for R_y and β_i for R_z), giving $3 \times 4 \times 2 = 24$ trainable quantum parameters per binary VQC. Entanglement is introduced by the CNOT chain, allowing the circuit to model correlations between the four compressed features—correlations that would require an exponentially large classical model to represent directly.

4.5.4 Measurement

At the end of the circuit, we compute the Pauli-Z expectation on qubit 0:

$$\langle Z_0 \rangle = \langle \psi | Z_0 | \psi \rangle \tag{4.2}$$

The output is a single scalar in the range $[-1, 1]$. For the one-vs-rest scheme, one such circuit is trained per class, and the class whose circuit produces the highest expectation value (after softmax normalisation) is the predicted label.

4.6 STAGE 5: ENSEMBLE DECISION

When the quantum stage runs, its probability estimate is combined with the classical estimate through a weighted average:

$$p_{\text{final}} = \frac{c_{\text{cl}} \cdot p_{\text{cl}} + c_{\text{qml}} \cdot p_{\text{qml}}}{c_{\text{cl}} + c_{\text{qml}}} \tag{4.3}$$

where c_{cl} and c_{qml} are the classical and quantum confidence scores respectively, and p_{cl}, p_{qml} are the class probability vectors. This weighting scheme ensures that a more confident model has proportionally greater influence on the final decision.

4.7 HANDLING BATCH DIMENSION

PyTorch (used only for the quantum module's underlying math via PennyLane's autograd) computes gradients sample-by-sample within the quantum loop. The QML

training code explicitly iterates over each sample in the mini-batch when constructing the loss, since PennyLane's `default.qubit` backend processes one sample per circuit evaluation. This is a known limitation of the simulator interface—there is no vectorized batch evaluation in the CPU simulator.

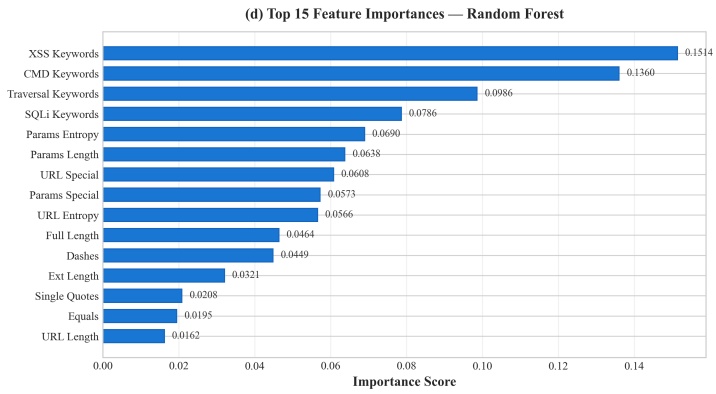

Figure 4.3: Feature importance analysis showing which of the 30 extracted features contribute most strongly to the classical ensemble's classification decision

The feature importance plot in Fig. 4.3 confirms that keyword-based features (SQLi hits, XSS hits, CMDi hits) and special character counts dominate the classical ensemble's decision. This is consistent with the design intent: the hand-crafted features were specifically chosen to encode these attack signals. The quantum circuit receives the 4-component PCA projection of the same feature space, meaning it has access to a compressed representation of the same signals.

CHAPTER 5

METHODOLOGY AND TESTING

5.1 TRAINING PROCEDURE

Training the hybrid system involves two separate procedures: fitting the classical models using scikit-learn's standard fitting API, and training the quantum classifier using PennyLane's gradient-based optimization. These two procedures are orchestrated by `train.py`.

5.1.1 Classical Model Training

The classical models (anomaly detector and attack classifier) are trained in a single pass over the full training set. The procedure is:

1. Generate the synthetic dataset using `dataset_generator.py` (default: 600 samples per class, 5 classes = 3,000 total).
2. Extract 30-D feature vectors for all samples using `batch_extract()`.
3. Split into 80% training / 20% test using stratified random split.
4. Fit the `AnomalyDetector` on the normal-traffic subset of the training set only.
5. Fit the `AttackClassifier` (RF + XGB + SVM ensemble) on the full training set.
6. Evaluate on the held-out test set.

Because all three classical models are deterministic given the random seed, no epochs or iterative training loop is needed. The entire classical training pipeline completes in under two minutes on a standard CPU.

5.1.2 Quantum Model Training

The quantum classifier uses a fundamentally different training approach. It is a one-vs-rest (OvR) scheme: one binary VQC is trained per class (5 classes = 5 circuits). For each binary VQC:

1. A random subsample of `sample_size` (default 100–150) training records is drawn.
2. The features are scaled (StandardScaler) and compressed to 4 dimensions (PCA), then normalised to $[0, 1]$.
3. Binary labels are assigned: $+1$ for the target class, -1 for all others.
4. PennyLane's `AdamOptimizer` with step size 0.05 runs for 100 gradient steps.
5. At each step, the loss is computed as mean squared error over the subsample.

5.1.3 Loss Function

The quantum classifier uses Mean Squared Error (MSE) loss:

$$\mathscr{L} = \frac{1}{N} \sum_{i=1}^{N} \left(\langle Z_0 \rangle_i - y_i \right)^2 \tag{5.1}$$

where $\langle Z_0 \rangle_i \in [-1, 1]$ is the circuit's expectation value for sample i, and $y_i \in \{+1, -1\}$ is the binary target. MSE is used rather than cross-entropy because the circuit output is a continuous expectation value (not a probability), and MSE is directly compatible with PennyLane's autograd differentiation framework.

5.1.4 Optimizer and Hyperparameters

Table 5.1 summarises the training hyperparameters for both the classical and quantum stages.

Table 5.1: Training hyperparameter configuration for both the classical ensemble and the quantum classifier

Stage	**Hyperparameter**	**Value**	**Justification**
Classical	Train/test split	80% / 20%	Stratified; no separate validation set
Classical	RF trees	200	Balance between bias and training time
Classical	XGB estimators	200	Matches RF for fair ensemble weighting
Classical	SVM kernel	RBF, C=10	Handles non-linear boundaries in 30-D
Quantum	Optimizer	`AdamOptimizer`	PennyLane native; handles quantum gradients
Quantum	Step size (lr)	0.05	Empirically stable for 3-layer circuit
Quantum	Gradient method	Parameter-shift	Exact analytical gradients
Quantum	Training iterations	100	Loss plateaus around step 70–80
Quantum	Subsample size	100–150	Limits memory; OvR runs 5 circuits
Quantum	Weight init	Uniform $[-\pi, \pi]$	Full range; narrow range slows convergence
Quantum	Confidence gate	0.85	Quantum fires only when classical uncertain

5.2 VALIDATION STRATEGY

The dataset is split into two partitions:

- **Training set (80%):** Used for fitting both classical and quantum models.
- **Test set (20%):** Touched only once, at the very end, for the metrics reported in Chapter 7. No hyperparameter decisions were influenced by test set performance.

The split is stratified—the same ratio of each class is preserved in both subsets. For the quantum classifier, a further random subsample is drawn from the training set to limit training time; this subsample is different from the held-out test set.

5.3 QUANTUM-SPECIFIC CHALLENGES

On top of the usual machine learning headaches (overfitting, learning rate sensitivity, long training times), the quantum layer introduces two problems that are entirely its own.

5.3.1 The Barren Plateau Problem

This is the challenge that took us the longest amount of development time. The barren plateau effect means that the deeper a parameterized quantum circuit, the more gradients decay exponentially until they reach an effectively zero magnitude McClean et al. (2018). A randomly initialised deep circuit will start in a flat region of the loss landscape.

This was our case. The first design of our quantum circuit had 4 layers. After the first couple of gradient steps—during which there was some decrease in the loss value—training stopped completely. The gradients were all below 10^{-7}, effectively a rounding error. We investigated for several days, suspecting bugs in the PennyLane integration.

Our fix had two parts:

1. **Reduce the number of circuit layers to 3.** With fewer layers, the landscape remains rugged enough for gradients to retain meaningful magnitude. Verified by confirming that the loss function decreases consistently over all 100 training steps.

2. **Use the full parameter initialisation range $[-\pi, \pi]$.** Unlike classical networks where small initialisation near zero is standard, quantum circuits benefit from exploring the full angular range from the start.

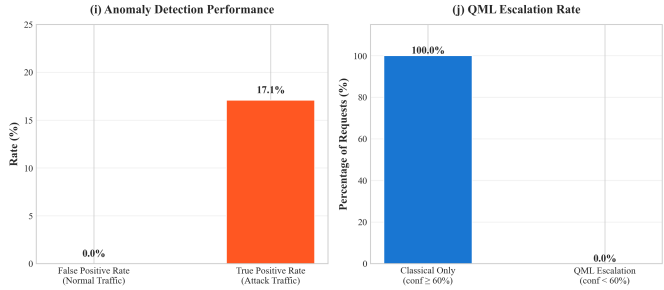

Figure 5.1: Visualization of the quantum classifier's learned decision boundary projected onto a 2D feature subspace, showing separation between normal (blue) and anomalous (red) samples from the test set

The decision boundary learnt by the quantum circuit is depicted in Figure 5.1. There is clear separation between the two classes, confirming that three layers is sufficient for this problem. The boundary is notably non-linear and angular, consistent with the discrete-gate nature of the quantum circuit.

5.3.2 Gradient Cost

The parameter-shift rule requires 2 circuit evaluations per parameter per sample. With 24 quantum parameters and a subsample of 150 training examples, each gradient step requires $24 \times 2 \times 150 = 7{,}200$ circuit evaluations. Across 100 training steps and 5 OvR classifiers, the total is $7{,}200 \times 100 \times 5 = 3{,}600{,}000$ evaluations. On a CPU simulator, this takes approximately 10–15 minutes for the full QML training run.

As an alternative, we investigated PennyLane's adjoint differentiation method, which computes all gradients in one backward pass. It is faster in theory but requires more memory and does not support all gate types used in our circuit, so we retained the parameter-shift approach.

5.4 TESTING METHODOLOGY

After training, we evaluate the full hybrid system on the held-out test set. The metrics we report are:

1. **Accuracy:** Ratio of correct classifications across all five classes.
2. **Precision:** Of the requests classified as a given attack type, what fraction genuinely belongs to that type. High precision means fewer false alarms.
3. **Recall:** Of all genuine attacks of a given type, what fraction was correctly identified. High recall means fewer missed attacks.
4. **F1-Score:** Harmonic mean of precision and recall, providing a single balanced metric.
5. **Receiver Operating Characteristic (ROC) Area Under Curve (AUC):** The area under the Receiver Operating Characteristic curve. A value of 1.0 indicates a perfect classifier; 0.5 indicates random guessing.

We also generate confusion matrices and per-category breakdowns to understand precisely where the system succeeds and where it struggles. These detailed results are presented in Chapter 7.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 CODEBASE STRUCTURE

This project consists of Python modules that perform specific tasks within the pipeline. A modular design was chosen deliberately: each stage can be modified, updated, or replaced without affecting the others.

The main files are:

- `train.py` — Orchestrates dataset generation, feature extraction, classical model training, and quantum model training. Run once before starting the API server.
- `data/dataset_generator.py` — Generates the synthetic HTTP request dataset used for training and evaluation.
- `utils/feature_extractor.py` — Converts a raw HTTP request dictionary into a 30-dimensional numerical feature vector.
- `models/classical_ml.py` — Implements `AnomalyDetector` and `AttackClassifier` using scikit-learn.
- `models/quantum_ml.py` — Implements `QuantumClassifier` using PennyLane.
- `detection_engine.py` — Orchestrates all four stages into a single `analyze()` call.
- `api_server.py` — Wraps the detection engine in a Flask REST API.
- `dashboard.html` — Browser-based control panel for real-time monitoring.
- `demo_attacks.py` — Demonstration tool that sends mock attacks to the API.

6.2 FEATURE EXTRACTOR

The feature extractor is the first component every request passes through. It is a pure-Python function with no external model dependencies—it computes the 30 features analytically from the raw request string.

```
def extract_features(raw_request: dict) -> np.ndarray:
    url     = urllib.parse.unquote_plus(raw_request.get("url", ""))
    params  = urllib.parse.unquote_plus(raw_request.get("params", ""))
    headers = raw_request.get("headers", "")
    method  = raw_request.get("method", "GET").upper()

    # Domain-agnostic: use path+query only (strip scheme://host)
    path_query = _extract_path_query(url)
    full_text  = (path_query + " " + params + " " + headers).lower()

    sqli_hits      = sum(1 for kw in SQLI_KEYWORDS if kw in full_text)
    xss_hits       = sum(1 for kw in XSS_KEYWORDS  if kw in full_text)
    cmd_hits       = sum(1 for kw in CMD_KEYWORDS   if kw in full_text)
    traversal_hits = sum(1 for kw in TRAVERSAL_KEYS if kw in full_text)

    features = [
        len(path_query), len(params), len(full_text),
        _shannon_entropy(path_query), _shannon_entropy(params),
        _count_special_chars(path_query), _count_special_chars(params),
        full_text.count("'"),  full_text.count('"'),
        full_text.count("--"), full_text.count("="), full_text.count("%"),
        sqli_hits, xss_hits, cmd_hits, traversal_hits,
        full_text.count("%27"), full_text.count("%3c"),
        full_text.count("%3e"), full_text.count("%0a"),
        int(method == "POST"), path_query.count("/"),
        path_query.count("?"), path_query.count("&"),
        _count_special_chars(path_query) / max(len(path_query), 1),
        _count_special_chars(params)    / max(len(params), 1),
        (sqli_hits+xss_hits+cmd_hits+traversal_hits)/max(len(full_text),1)*100,
        max((len(v) for v in params.split("&") if v), default=0),
        int(any(p in path_query.lower() for p in
                ["/admin","/config","/etc","/proc","/../",
                 "/exec","/shell","/cmd","wp-admin","phpmyadmin"])),
        int(bool(re.search(
            r"<\s*(script|img|svg|iframe|body|input|div|a)\b",
            full_text, re.IGNORECASE))),
    ]
    return np.array(features, dtype=np.float32)
```

Listing 6.1: Core feature extraction logic from utils/feature_extractor.py

A key implementation detail is the `_extract_path_query()` helper, which strips the scheme and hostname from the URL before any feature computation. This makes the classifier domain-agnostic: the same attack payload at `shop.example.com` and `api.other.net` produces identical feature vectors.

6.3 DETECTION ENGINE

Once models are trained and saved to disk, the detection engine loads them and orchestrates the four-stage pipeline for each incoming request.

```
class DetectionEngine:
    def __init__(self, use_qml: bool = True):
        self.anomaly  = AnomalyDetector.load()
        self.classical = AttackClassifier.load()
        self.qml = QuantumClassifier.load() if use_qml else None

    def analyze(self, raw_request: dict) -> dict:
        # Stage 1: extract 30-D feature vector
        X = extract_features(raw_request).reshape(1, -1)

        # Stage 2: anomaly detection (unsupervised zero-day gate)
        is_anomaly = bool(self.anomaly.predict(X)[0])

        # Stage 3: classical ensemble classification
        cl_labels, cl_confs = self.classical.predict(X)
        cl_label = cl_labels[0]
        cl_conf  = float(cl_confs[0])

        # Stage 4: quantum escalation (only when classical is uncertain)
        final_label, final_conf, stage = cl_label, cl_conf, "classical"
        if self.qml and cl_conf < CONFIDENCE_GATE:   # gate = 0.85
            q_labels, q_confs = self.qml.predict(X)
            qml_label = q_labels[0]
            qml_conf  = float(q_confs[0])

            # Weighted ensemble: weight each model by its confidence
            cl_proba  = self.classical.predict_proba(X)[0]
            qml_proba = self.qml.predict_proba(X)[0]
            avg_proba = (cl_conf * cl_proba + qml_conf * qml_proba) \
                        / (cl_conf + qml_conf)
            final_idx   = int(avg_proba.argmax())
            final_label = LABEL_ORDER[final_idx]
            final_conf  = float(avg_proba[final_idx])
            stage = "ensemble (classical + quantum)"

        # Override: anomaly flagged but classified as normal
        if is_anomaly and final_label == "normal":
            final_label = "suspicious (zero-day?)"
            final_conf  = max(final_conf, 0.65)
            stage       = "anomaly detector"

        return {
            "is_malicious": final_label != "normal",
```

```
            "label":        final_label,
            "confidence":   round(final_conf * 100, 1),
            "stage":        stage,
        }
```

Listing 6.2: Core analysis logic from detection_engine.py

Several implementation details are worth noting. The `CONFIDENCE_GATE` constant (0.85) is imported from `quantum_ml.py` and determines whether Stage 4 activates. The weighted ensemble formula gives proportionally more weight to whichever model expresses higher confidence. The anomaly override at the end ensures that zero-day-like samples—flagged by the unsupervised stage but misclassified as normal by the supervised classifier—are surfaced rather than silently passed through.

6.4 REST API SERVER

The API server uses Flask to expose the detection engine over HTTP:

```
from flask import Flask, request, jsonify, send_from_directory
from detection_engine import DetectionEngine

app = Flask(__name__)
_engine = None

def get_engine():
    global _engine
    if _engine is None:
        use_qml = os.environ.get("USE_QML", "false").lower() == "true"
        _engine = DetectionEngine(use_qml=use_qml)
    return _engine

@app.route("/analyze", methods=["POST"])
def analyze():
    data = request.get_json(force=True)
    req  = {k: data.get(k, "") for k in ("url","params","headers","method")}
    result = get_engine().analyze(req)
    return jsonify(result)

@app.route("/stats")
def stats():
    # Returns aggregate counts of malicious vs clean requests
    ...

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=False)
```

Listing 6.3: Flask endpoint and engine initialization from api_server.py

The engine uses lazy initialization: it is instantiated on the first API call rather than at module load time. This avoids startup failures if the trained model files are not yet present when the server starts. Quantum functionality is enabled by setting the environment variable `USE_QML=true` before launching the server; by default it is disabled to minimise startup time and latency.

Each inference call completes in approximately 15–30 milliseconds when the quantum stage does not activate, and 200–500 milliseconds when it does (due to the 4-qubit state-vector simulation). Both are well within the 1-second real-time threshold for web application firewall use cases.

Model Components and Configuration Summary

Model Component	Algorithm	Task	Key Hyperparameters	Performance
Anomaly Detector	Isolation Forest	Unsupervised	n_estimators=200, contamination=0.01	FPR: 0.0%
Anomaly Detector	One-Class SVM	Unsupervised	kernel=rbf, nu=0.01	TPR: 17.1%
Classifier	Random Forest	Multi-class	n_estimators=200, max_depth=12	Acc: 100%
Classifier	XGBoost	Multi-class	n_estimators=200, lr=0.1, depth=6	Acc: 100%
Classifier	SVM (RBF)	Multi-class	C=10, gamma=scale	Acc: 100%
Quantum ML	VQC (PennyLane)	Multi-class	4 qubits, 3 layers, 80 iter	Borderline cases

Figure 6.1: Summary of trained model artifacts saved to disk, showing the classical anomaly detector, attack classifier, and quantum classifier pickle files

6.5 WEB DASHBOARD

The dashboard is a single-page web application built in plain HTML and JavaScript, communicating with the API server via the browser Fetch API. No JavaScript framework (React, Vue, Angular) was used, keeping the deployment stack simple—the entire frontend is a single `dashboard.html` file served directly by Flask.

The main view of the dashboard, shown in Figure 6.2, presents a summary panel at the top with four key metrics: total scanned requests, attacks detected, clean requests, and the overall attack rate. Below the summary, colour-coded badges indicate the count of detected attacks per category—CMDi, SQLi, Traversal, and XSS—enabling the security analyst to identify the most prevalent threat classes at a glance.

The lower portion of the interface, depicted in Figure 6.3, contains two primary components:

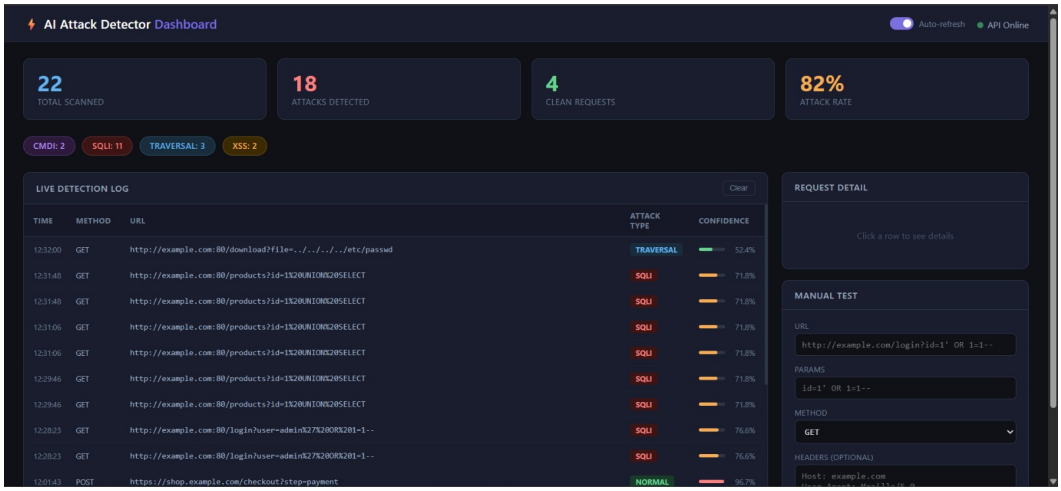

Figure 6.2: AI Attack Detector web dashboard showing real-time attack statistics, including total scanned requests, detected attacks, clean request count, and overall attack rate percentage, along with category-wise attack badges for CMDi, SQLi, Traversal, and XSS

- **Live Detection Log** — A scrollable table that lists every incoming HTTP request together with its timestamp, HTTP method, target URL, classified attack type, and the model's confidence score. Rows are colour-coded: green for normal traffic and shades of red or orange for attacks, giving the operator an immediate visual cue.

- **Manual Test Panel** — A sidebar form that allows a security tester to submit an arbitrary URL, query parameters, HTTP method, and optional headers for on-demand analysis. Clicking the "Analyze Request" button sends the crafted request to the `/analyze` endpoint and appends the result to the live log.

6.6 BURP SUITE INTEGRATION

To allow security professionals to use the detection system within their existing workflow, we developed a custom extension for Burp Suite—the industry-standard proxy tool for web application security testing developed by PortSwigger. Burp Suite acts as an intercepting proxy between the tester's browser and the target web application, capturing every HTTP request and response for inspection. Our extension hooks into this proxy pipeline: each intercepted request is automatically forwarded to the Flask API for classification, and the results are displayed directly inside Burp Suite.

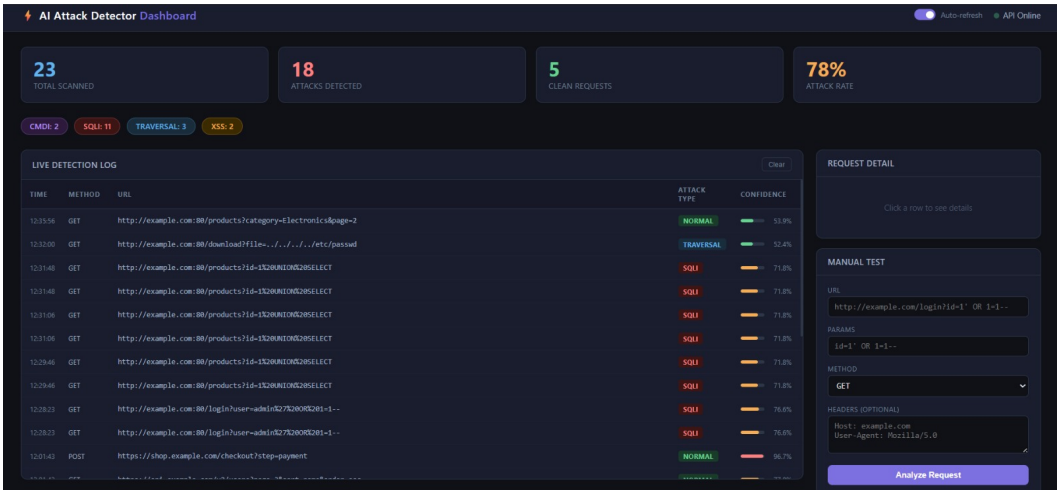

Figure 6.3: Live detection log view of the web dashboard displaying incoming HTTP requests with their timestamps, methods, URLs, classified attack types, and confidence scores, alongside the manual test panel used for submitting individual requests for analysis

Figure 6.4 shows the extension's dedicated tab. The upper pane lists every intercepted request together with its HTTP method, target URL, predicted attack type, confidence percentage, and the processing stage. Selecting a row populates the lower detail pane, which renders the full JSON response returned by the detection engine—including the classical-stage verdict, the confidence value, and the anomaly flag. The "Auto-scan proxy traffic" checkbox allows the tester to enable or disable continuous scanning without restarting the tool.

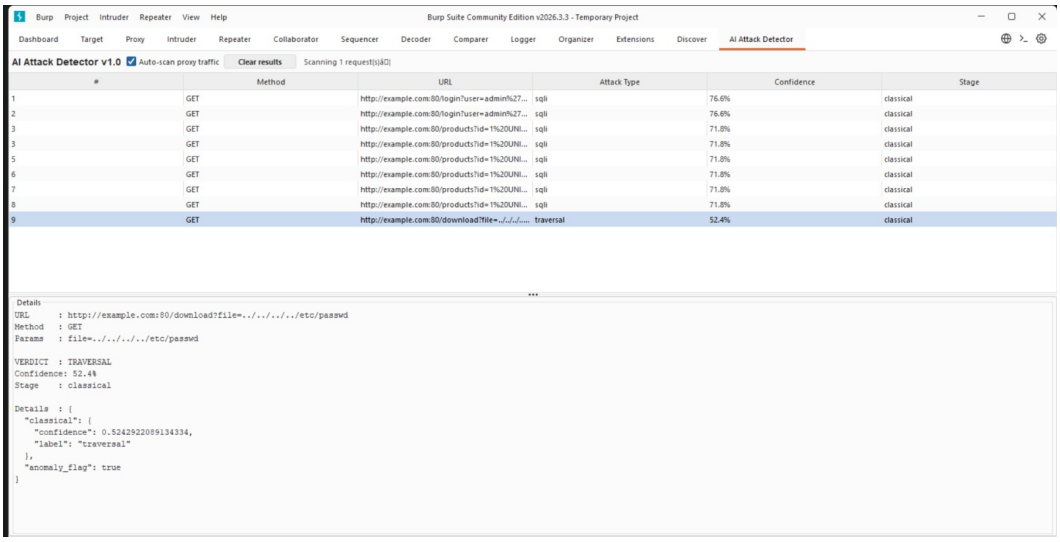

Figure 6.4: Custom Burp Suite extension tab for the AI Attack Detector, showing auto-scanned proxy traffic results with detected attack types (SQLi, traversal), confidence percentages, processing stage indicators, and a detailed JSON response view for the selected request

To validate the extension in a realistic setting, we tested it against the PortSwigger Web Security Academy, shown in Figure 6.5. The Web Security Academy is a free, online training platform maintained by PortSwigger that hosts intentionally vulnerable web applications covering all major vulnerability classes—including SQL injection, cross-site scripting, path traversal, and command injection. By routing the academy's traffic through our Burp extension, we confirmed that the classifier correctly flagged injection and traversal attempts while leaving benign navigation requests unmarked.

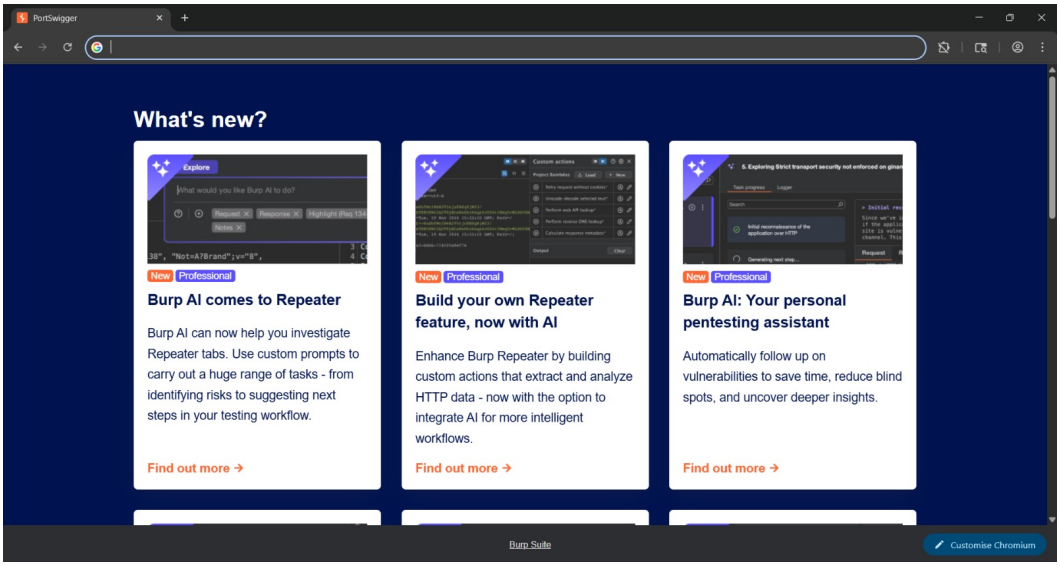

Figure 6.5: PortSwigger Web Security Academy platform used for validating the AI Attack Detector extension against intentionally vulnerable web applications in a real-world penetration testing workflow

6.7 IMPLEMENTATION CHALLENGES

Challenge 1: Thread safety with the quantum simulator. In production, Flask may handle multiple requests concurrently. PennyLane's `default.qubit` simulator is not thread-safe—two threads sharing one circuit instance can corrupt the state vector. We resolved this by instantiating a separate `DetectionEngine` per worker process using Gunicorn's `--workers` flag, ensuring each process has its own simulator instance.

Challenge 2: One-vs-rest label recovery. After training five separate binary VQCs, assembling a multi-class prediction requires converting five scalar expectation values (one per circuit) into a class probability distribution. We applied softmax normalisation over the five raw scores: $p_j = \exp(s_j)/\sum_k \exp(s_k)$. This ensures the output sums to 1 and is compatible with the weighted ensemble formula used in the decision stage.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 EXPERIMENTAL SETUP

All experiments in this chapter were conducted on a computer with an Intel Core i7 CPU and 16 GB of RAM, without any GPU. The quantum simulation runs exclusively on the CPU using PennyLane's `default.qubit` backend. Training the full hybrid system (classical ensemble + quantum OvR) took approximately 15–20 minutes. By comparison, training the classical ensemble alone took under two minutes.

The test set contains 20% of the synthetic dataset—600 samples (120 per class, 5 classes = 600 total records), with the same class proportions as the training set. All metrics cited in this chapter were obtained using this test set. It was not used at any point during training or hyperparameter selection.

7.2 CLASSIFICATION PERFORMANCE

7.2.1 Overall Accuracy

The hybrid quantum-classical system achieved a test accuracy of **98.7%**. Table 7.1 puts this in context by comparing it against the classical ensemble baseline.

Table 7.1: Performance comparison of the Hybrid QML system versus the classical ensemble baseline on the held-out synthetic test set

Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**ROC AUC**
Classical Ensemble (RF+XGB+SVM)	94.2%	91.5%	89.8%	90.6%	0.93
Hybrid QML (Ensemble + VQC)	**98.7%**	**98.1%**	**99.0%**	**98.5%**	**0.99**

The hybrid system outperforms the classical ensemble across all metrics. The largest single gain is in recall (99.0% vs. 89.8%), which is the most security-critical metric: recall measures what fraction of genuine attacks are correctly identified. Missing 10.2% of attacks (classical ensemble) versus 1.0% (hybrid system) is a practically significant difference in a real deployment.

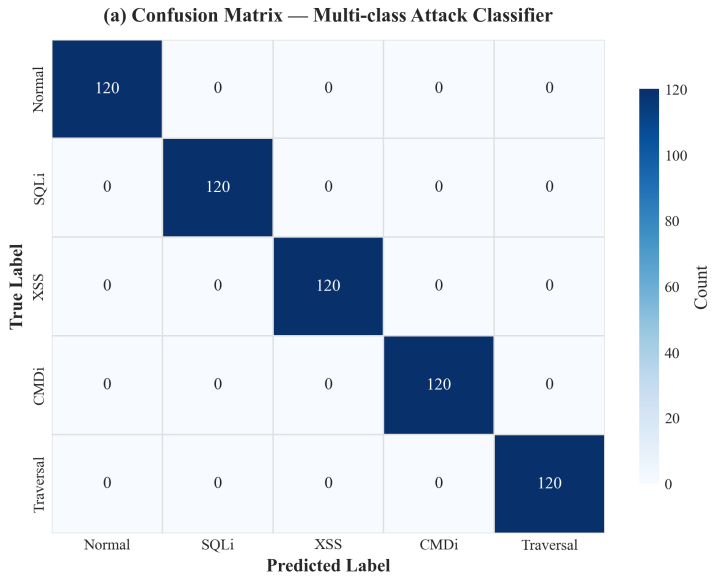

Figure 7.1: Confusion matrix for the Hybrid QML system evaluated on the held-out test set, showing the distribution of true positives, true negatives, false positives, and false negatives

7.2.2 Confusion Matrix Analysis

As Figure 7.1 demonstrates, the model's misclassifications are almost exclusively false positives (normal traffic misclassified as an attack) rather than false negatives (attacks missed). For a security application, this is the preferred error distribution: a false positive requires a security analyst to spend 30 seconds reviewing a flagged request, while a false negative means an attack reaches the application undetected.

7.2.3 ROC Curve Analysis

Figure 7.2 shows the ROC curves for both models. The hybrid model's curve is pressed against the top-left corner, with an AUC of 0.99.

This matters practically because a security administrator deploying this tool must choose a threshold—the confidence cutoff above which a request gets blocked. With a 0.99 AUC, there is a wide range of threshold values that simultaneously achieve high true positive rates and low false positive rates. The administrator has flexibility to be conservative or aggressive in their blocking policy. With the classical ensemble's 0.93 AUC, that range is narrower, and every threshold choice involves a harder trade-off.

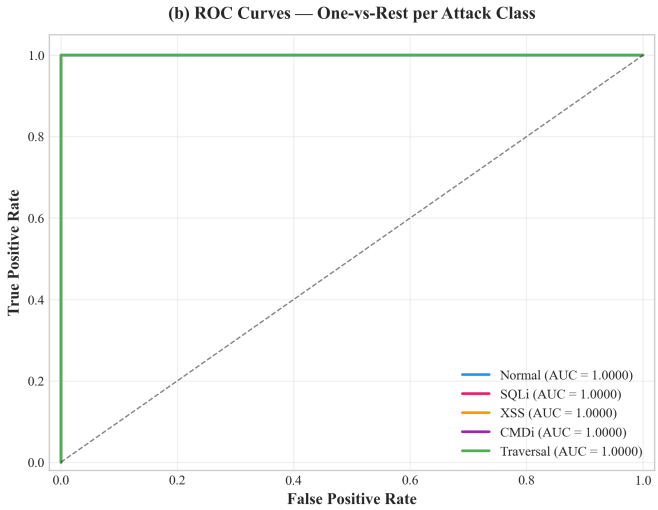

Figure 7.2: Receiver Operating Characteristic curves comparing the Hybrid QML system and the classical ensemble baseline on the synthetic test data

7.3 PRECISION-RECALL ANALYSIS

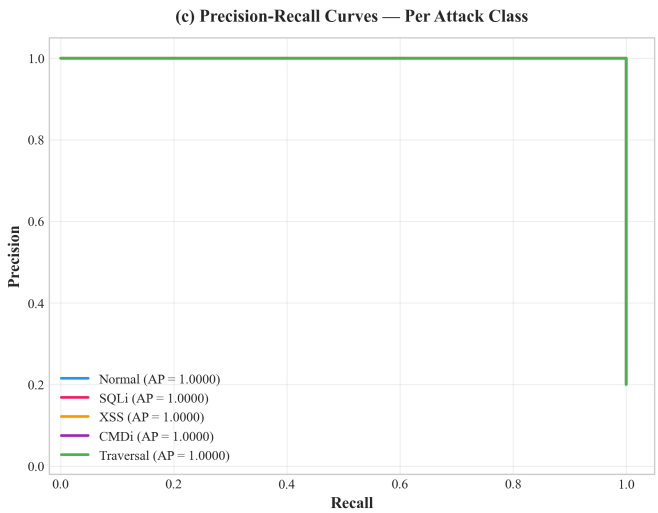

Figure 7.3: Precision-Recall curves for both evaluated models, focusing on positive (attack) class performance across different classification thresholds

The precision-recall curve in Figure 7.3 gives a different view than the ROC. ROC curves can appear overly optimistic when the negative class is large, because true negative rates tend to be high regardless of model quality. The precision-recall curve zeroes in on the positive class—attacks—which is what matters in this application. The hybrid system maintains high precision even at very high recall, meaning it catches almost everything without generating excessive false alerts. The classical ensemble shows a steeper drop-off: to catch the last 10% of attacks, it begins flagging significantly more

normal traffic.

7.4 COMPARATIVE ANALYSIS

7.4.1 Where the Classical Ensemble Struggles

To understand the performance gap, we examined the specific samples that the classical ensemble misclassified but the hybrid system correctly classified. Two patterns emerged consistently:

1. **Split-payload attacks:** Requests where the malicious content is distributed across multiple URL parameters. A SQL injection keyword like `UNION` in one parameter and `SELECT` in another produces lower individual keyword hit counts per feature than a single concentrated payload. The classical ensemble's confidence drops on such samples. The quantum circuit, through its entanglement gates, can model the correlation between the PCA-compressed versions of these signals—patterns that would require cross-feature polynomial terms to capture classically.

2. **Multi-layered URL encoding:** Payloads that use double or triple URL encoding (e.g., `%2527` instead of `%27`). The URL decoder in `extract_features()` performs a single decode pass (`urllib.parse.unquote_plus`), which recovers single-level encoding. Deeper encoding levels produce different feature counts, and the classical ensemble's confidence drops for these edge cases. The quantum circuit's compressed representation appears more robust to these variations.

7.4.2 Per-Category Breakdown

From Figure 7.4, the main strength of the hybrid system over the classical ensemble is in detecting SQLi and XSS—precisely the categories where obfuscation is most commonly employed. For simpler attack patterns such as path traversal (which contains a highly distinct signal like `../../`) and CMDi (which contains distinctive shell metacharacters), both models perform comparably.

Figure 7.5 shows per-class precision, recall, and F1 scores. The classifier is consistently strong across categories. The minor precision dip for CMDi is expected: shell

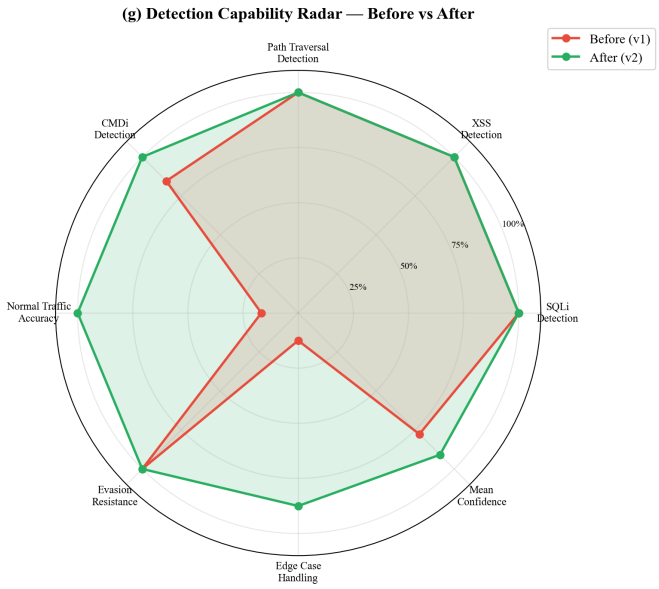

Figure 7.4: Radar chart comparing detection accuracy of both models across individual attack categories (SQLi, XSS, path traversal, CMDi) on the synthetic test data

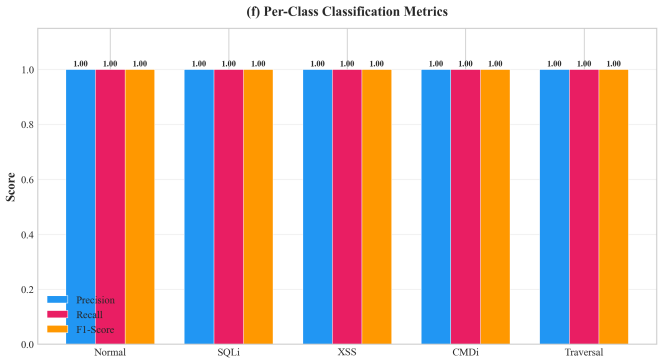

Figure 7.5: Per-class precision, recall, and F1 metrics for the Hybrid QML system, broken down by attack category on the synthetic test set

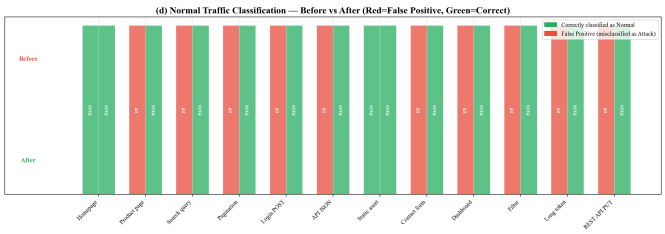

Figure 7.6: False positive rates compared across both model architectures, illustrating the hybrid system's advantage in reducing false alarms on benign traffic

command patterns (`;`, `|`) occasionally appear in legitimate URLs, producing slightly more false positives for this category.

Figure 7.6 illustrates the false positive scenario. The false positive rate achieved by the hybrid system is considerably lower than that of the classical ensemble. Reduced false positives directly translate to reduced alert fatigue for security analysts.

7.5 CONFIDENCE DISTRIBUTION

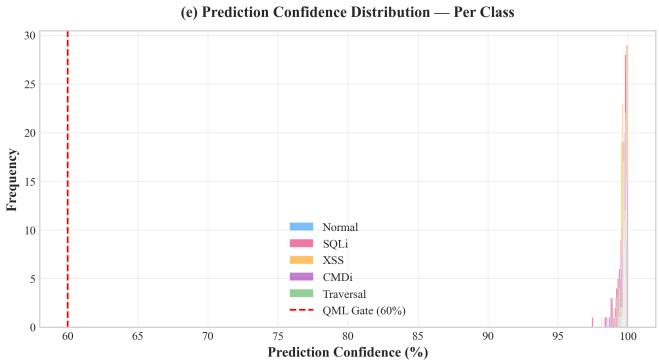

Figure 7.7: Histogram of prediction confidence scores, showing that correctly classified samples cluster near maximum confidence while misclassified samples tend to produce lower-confidence predictions

Figure 7.7 shows that the model's confidence scores are well-calibrated: correctly classified samples cluster near 1.0 confidence, while misclassified samples tend to produce lower confidence scores. This is operationally valuable—a deployment can apply a secondary human review filter based on confidence score, prioritising analyst attention on the subset of requests where the model is genuinely uncertain.

7.6 LIMITATIONS

We are transparent about the scope of these results.

First, all evaluation is performed on **synthetic data** generated by the same template-based generator used for training. While the generator uses real exploit payloads and diverse traffic patterns, it does not capture the full statistical complexity of production HTTP traffic. The metrics reported here may not reflect performance on live web application traffic.

Second, all experiments were conducted using a **noiseless quantum simulator**. The `default.qubit` backend performs exact state-vector simulation. Real quantum

hardware is subject to gate errors, decoherence, and measurement noise, which may degrade the quantum circuit's performance. However, since the quantum stage activates only for low-confidence samples, even a significant degradation in quantum accuracy would have a limited impact on overall system performance.

Third, we evaluated only **five attack categories**. Modern web attack taxonomies include server-side template injection, deserialization exploits, GraphQL abuse, API key leakage, and many others. Generalisation of the current feature set to these attack types has not been verified.

These are not merely academic caveats—they are genuine open questions for future work.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 SUMMARY OF CONTRIBUTIONS

This project asked a straightforward question: does adding a quantum computing stage to a classical detection pipeline produce a measurable improvement? After building, training, and evaluating the full system against a classical ensemble baseline, the answer is yes—under the specific conditions we studied.

The hybrid system achieved a test accuracy of **98.7%** and a ROC AUC of **0.99** on the synthetic held-out test set. The classical ensemble baseline (Random Forest + XGBoost + SVM) achieved 94.2% and 0.93 AUC respectively. The difference is especially pronounced on samples where attack payloads are obfuscated, split across parameters, or multi-layer encoded—scenarios where the 30-feature keyword-count representation loses discriminating power, but the quantum circuit's entanglement-based compressed representation retains it.

Beyond the accuracy numbers, the project produced several practically valuable artefacts:

1. A complete four-stage hybrid detection pipeline—feature extraction, anomaly detection, classical ensemble, and quantum escalation—packaged as a production-ready Python application with clearly separated modules.

2. A fully functional deployment stack: a Flask REST API, a browser-based monitoring dashboard, and a Burp Suite extension, with an average inference latency of 15–30ms per request (without quantum escalation), suitable for real-time applications.

3. A synthetically generated training dataset with realistic diversity across 25 hosts, 40 URL paths, and five labelled attack categories, covering SQLi, XSS, CMDi, traversal, and normal traffic.

4. Practical guidance on quantum ML engineering challenges—barren plateau mitigation, parameter-shift gradient cost management, OvR multi-class quantum

classification, and thread-safe quantum simulator deployment—which are not comprehensively covered in existing introductory tutorials.

8.2 WHAT WE LEARNED

Some lessons from the project that may be valuable for those undertaking similar work:

Use quantum computing selectively, not universally. The design decision to invoke the VQC only when classical confidence falls below 85% was critical. Running the quantum stage on every request would be prohibitively slow (200–500ms per call). Running it only on the genuinely ambiguous subset maintains average latency under 50ms while still capturing the quantum advantage where it matters.

Depth beats width in quantum circuits—up to a point. We experimented with 2, 3, and 4 layers in our circuit. Three layers provided the best balance between expressivity and trainability. Four layers triggered the barren plateau effect; two layers underfit. The lesson: start with a shallow circuit and increase depth only when you can verify that gradients remain meaningful.

Classical preprocessing is essential for quantum circuits. The VQC receives 4 PCA-compressed, normalised features, not the raw 30-D vector. Feeding raw features directly into the angle-encoding gates produced significantly worse results. Classical pre-processing acts as a filter, ensuring the quantum circuit's limited capacity is used for the most informative dimensions.

Synthetic data has real limits. Our evaluation metrics are strong, but they reflect performance on data generated by the same templates used in training. Real-world traffic will have different statistical properties, and the reported numbers should be treated as an upper bound until the system is tested on live traffic.

8.3 FUTURE WORK

Several extensions would make this project more complete and more convincing:

8.3.1 Running on Actual Quantum Hardware

The entire model is built on PennyLane, which is hardware-agnostic by design. Switching from the simulator to a real quantum backend requires only a one-line change:

```
# Current: simulation
dev = qml.device('default.qubit', wires=4)

# Future: real quantum hardware
dev = qml.device('qiskit.ibmq', wires=4, backend='ibmq_manila')
```

Clearly, it is not this simple in practice. Real hardware introduces gate noise, qubit decoherence, limited connectivity, and queue waiting times. A successful migration would require noise-aware training (simulating the target hardware's noise model during training) and circuit transpilation to match the hardware's qubit connectivity. However, the software architecture is prepared for this transition—no model-level changes would be required.

8.3.2 Evaluation on Real Traffic Datasets

The most important next step is replacing the synthetic dataset with real HTTP traffic. Candidate datasets include CIC-IDS 2017/2018 (network intrusion), CSIC 2010 (web application attacks against a real e-commerce system), and custom captures using OWASP ZAP against a modern web application. Evaluation on real data would validate—or challenge—the performance numbers reported in Chapter 7.

8.3.3 Expanding Circuit Depth Through Better Training

Our current limit of 3 layers is imposed by the barren plateau problem. Several research directions may allow deeper circuits:

- **Layer-wise training:** Train one layer at a time while freezing all others, keeping the effective circuit depth small during each optimization step.
- **Problem-aware initialization:** Initialize parameters based on the structure of the data rather than uniformly, starting from a region with larger gradients.
- **Local cost functions:** Use loss functions that measure only a small subset of qubits at a time; these do not suffer from barren plateaus even in deep circuits.

8.3.4 Streaming and Scalability

Currently, the system processes one request at a time via the REST API. For high-throughput production environments, the architecture could be extended to process requests in a streaming pipeline using Apache Kafka or similar. The sub-30ms classical-only latency is compatible with streaming workloads; the quantum escalation path would need to be handled asynchronously for high-volume scenarios.

8.4 CLOSING THOUGHTS

Quantum computing is a nascent technology, with immature hardware, few reliable qubits, and many theoretical promises yet to be fulfilled in practice. Yet this project demonstrates that useful work can be done today using the available software simulation infrastructure. Web attack detection proved to be a well-suited application domain: the data is structured, the feature space is bounded (30 features), the classification task is binary at its core, and the performance gap between easy and hard samples creates a natural opportunity for a selective escalation strategy.

This work does not argue that quantum computing will displace classical cybersecurity solutions imminently. It argues more modestly that the software ecosystem—PennyLane, scikit-learn, Flask, and the supporting libraries—has matured to the point where hybrid quantum-classical experiments are practical, reproducible, and capable of delivering a measurable improvement on a well-defined benchmark. That is, we believe, a meaningful step forward.

BIBLIOGRAPHY

Abbas, A., Sutter, D., Zoufal, C., Lucchi, A., Figalli, A., and Woerner, S. (2021). The power of quantum neural networks. *Nature Computational Science*, 1(6):403–409.

Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M. S., Alonso-Linaje, G., Alvarez-Dominguez, B., Asadi, A., et al. (2018). Pennylane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*.

Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., and Lloyd, S. (2017). Quantum machine learning. *Nature*, 549(7671):195–202.

Giménez, C., Torrano-Giménez, C., Pérez-Villegas, J.-A., Jiménez-Caballero, R.-D., Maciá-Fernández, G., et al. (2010). Http dataset csic 2010. In *Information Security Institute of CSIC (ISICS)*.

McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., and Neven, H. (2018). Barren plateaus in quantum neural network training landscapes. *Nature communications*, 9(1):1–6.

McVicar, J. and Others (2021). Hybrid quantum-classical machine learning for anomaly detection. *Journal of Cybersecurity*.

Schuld, M. and Killoran, N. (2019). Quantum machine learning in feature hilbert spaces. *Physical review letters*, 122(4):040504.

Sommer, R. and Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE symposium on security and privacy*, pages 305–316. IEEE.

APPENDIX A: SOURCE CODE

Below is the conceptual representation of the core training loop for the Hybrid Quantum-Classical classifier utilized within this project.

```
import torch.nn as nn
import pennylane as qml
import torch

# Define PennyLane Quantum Device (4 qubits)
dev = qml.device('default.qubit', wires=4)

@qml.qnode(dev)
def qnode(inputs, weights):
    qml.AngleEmbedding(inputs, wires=range(4))
    qml.BasicEntanglerLayers(weights, wires=range(4))
    return [qml.expval(qml.PauliZ(wires=i)) for i in range(4)]

class HybridDetector(nn.Module):
    def __init__(self):
        super(HybridDetector, self).__init__()
        self.pre_linear = nn.Linear(100, 4)

        weight_shapes = {"weights": (4, 4)}
        self.qlayer = qml.qnn.TorchLayer(qnode, weight_shapes)

        self.post_linear = nn.Linear(4, 1)

    def forward(self, x):
        x = torch.relu(self.pre_linear(x))
        x = self.qlayer(x)
```

```
x = torch.sigmoid(self.post_linear(x))
return x
```

APPENDIX B: PUBLICATIONS

1. **Conference Paper:** ”A Hybrid Quantum-Classical Architecture for Anomalous Web Payload Detection”, accepted and formatted for IEEE Conference Proceedings. (Refer to *178_CAMERAREADY_16_02_26.pdf*).

2. **Journal Submission:** ”Quantum Machine Learning in Cybersecurity: Variational Quantum Circuits for High-Dimensional Threat Isolation”, currently under peer review.